Laborator 1: Java Basics

1 Responsabil

Gabriel Guțu-Robu, gabriel.gutu@upb.ro

Publicat: 09 octombrie 2022, 09:00

Actualizat la: 10 octombrie 2022, 09:55 (adaugat tabel lipsa exercitiul 3)

2 Overview

Caracteristici Java:

- *Uşurinţă în utilizare şi învăţare*.
- *Independenţă de platformă*
- *Extensibilitate* ca biblioteci disponibile
- *Volum foarte mare de informaţii şi de resurse* disponibile pentru învăţare.
- *Robusteţe* – detectarea erorilor de către compilator şi gestionarea situaţiilor de excepţie
- *Alocarea memoriei* se face eficient prin intermediul unui mecanism de tip stivă
- *Rularea pe mai multe fire de execuţie* şi executarea mai multor task-uri simultan

2.1 API-ul Java

Ediţii Java:

- *Standard Edition* (SE; Oracle, n.d.a) – pentru dezvoltarea aplicaţiilor desktop şi server, inclusiv sub formă de abonament cu asistenţă tehnică: *SE –Subscription*;
- *Enterprise Edition* (EE; Oracle, n.d.b) – vine în sprijinul dezvoltatorilor din industrie inclusiv aplicaţii web;
- *Micro Edition* (ME; Oracle, n.d.c) – mediu robust şi flexibil pentru aplicaţiile ce rulează pe sisteme embedded sau dispozitive mobile în Internet of Things (microcontrollere, senzori, Personal Digital Assistants (PDAs), imprimante etc.);
- Versiuni specifice, de exemplu: *Card* (Oracle, n.d.d) – pentru aplicaţii ce rulează pe smart cards sau device-uri cu memorie sau capabilităţi de procesare limitate – *Java TV* (Oracle, n.d.e) – o versiune a lui Java ME ce implementează soluţii pentru sisteme TV sau mediabox-uri.

Java Development Kit (JDK) reprezintă o implementare a limbajului Java, o maşină virtuală alcătuită din Java Virtual Machine (JVM) şi instrumente utile pentru dezvoltarea şi testarea programelor scrise în Java şi care rulează pe platforma Java. Pe lângă JDK, trebuie folosit şi un mediu de dezvoltare integrat (**Integrated Development Environment – IDE**)

- NetBeans (Apache NetBeans, n.d.), Eclipse (Eclipse Foundation, n.d.), IntelliJ IDEA (Jetbrains, n.d.). Editarea, compilarea si rularea programelor Java sunt astfel integrate într-o intefață grafică unică.

2.2 COMPILAREA ȘI EXECUȚIA UNUI PROGRAM JAVA

Exemplu simplu de program Java, numit **Exemplu.java**:

```
1 public class Exemplu {
2     public static void main(String[] args) {
3           // Afișare mesaj la consola
4           System.out.println("Acesta este un exemplu!");
5     }
6 }
```

Linia 1 definește o clasă. Fiecare program Java trebuie să aibă cel puțin o clasă. Fiecare clasă are un nume. Prin convenție, numele clasei trebuie să înceapă cu literă mare. În acest exemplu, *numele clasei* este `Exemplu`. Linia 2 definește metoda `main`. Pentru a rula o clasă, aceasta trebuie să conțină o metoda numită `main`. Programul este executat din metoda `main`. Linia 3 conține un comentariu. Acestea pot fi de două tipuri: pe linie (//) sau în bloc (/* */). Pe Linia 4 este afișat un mesaj la consolă. Cuvintele cheie rezervate limbajului sunt `class`, `public`, `static`, `void`. Limbajul Java este *case sensitive*. Astfel, este greșit să se înlocuiască cuvântul `main` cu Main. Ca orice limbaj de programare, Java are propria sintaxă, iar utilizatorii trebuie să scrie cod ce respectă aceste reguli. Dacă ele sunt încălcate, compilatorul Java va raporta automat apariția erorilor de sintaxă.

După scrierea programului, acesta trebuie compilat înainte de a fi executat. Dacă programul are erori de compilare, acestea trebuie rezolvate și programul recompilat. Dacă programul are erori la rulare (runtime) sau nu furnizează rezultatul corect, el trebuie modificat, recompilat și executat din nou. Codul Java este scris într-un *fișier sursă*. Compilatorul traduce fișierul sursă într-un fișier *bytecode*. Următoarea comandă compilează `Exemplu.java`:

```
javac Exemplu.java
```

Înainte de compilare și rulare, trebuie instalat și configurat JDK-ul. Dacă nu sunt erori de sintaxă, compilatorul generează un fișier bytecode cu extensia `.class`. Astfel, pentru exemplul de mai sus, se generează fișierul `Exemplu.class`.

3 ELEMENTE SPECIFICE DE LIMBAJ

3.1 Identificatori

Identificatorul este o secvență de *litere, cifre, liniuța de subliniere* (_) *și caracterul dolar* (`$`), dintre care primul caracter nu trebuie să fie cifră. Un identificator poate fi `true`, `false`

sau `null`, de orice lungime și nu trebuie să se confunde cu un cuvânt rezervat limbajului. Identificatorii pot numi *variabile, constante, metode, clase și pachete*. Nu este recomandat totuși să se folosească `$` ca prim caracter.

3.2 Variabile

Variabilele sunt utilizate pentru a stoca valori ce se vor folosi mai târziu în program. Declararea unei variabile este o modalitate prin care se precizează compilatorului cantitatea de memorie pe care trebuie să o aloce în program, bazându-se pe tipul de date al variabilei. Declararea se face astfel:

```
datatype variableName;
```

Exemplu:

```
int count;
float salary;
double x, y, z;
```

Prin convenție, numele unei variable se scrie cu litere mici. Dacă numele variabilei este alcătuit din mai multe cuvinte concatenate, atunci se scrie cu literă mare prima literă începând de la al doilea cuvânt.

3.3 Atribuiri

După ce o variabilă este declarată, i se poate atribui o valoare utilizându-se operatorul de atribuire = , cu forma:

```
variabilă = expresie
```

O expresie este alcătuită din valori, variabile și operatori.

```
int a = 1;
x = 2 * y + 3 * z;
int count = a + 1;
```

În Java există noțiunea de *atribuire expresie*.

```
System.out.println(x = 1);
```

este echivalent cu

```
x = 1;
System.out.println(x);
```

Atribuirea `x = y = z = 2` este corectă și este echivalentă cu:

```
x = 2;
y = 2;
z = 2;
```

Într-o atribuire, tipul de date al variabilei din stânga trebuie să fie compatibil cu cel al variabilei din dreapta. De exemplu, atribuirea `int x = 1.0` este incorectă deoarece `x` este de tipul `int` iar `1.0` este de tipul `double`.

3.4 Constante

Valoarea unei variabile se schimbă de-a lungul execuției unui program, dar cea a unei constante nu. Pentru a declara o constantă se folosește sintaxa:

```
final datatype NUMECONSTANTA = VALOARE;
final double PI = 3.1415926;
```

`final` este un cuvânt cheie din Java care se utilizează pentru a declara o constantă. Constantele se scriu cu literă mare și au ca avantaj faptul că nu trebuie să se repete în scrierea programului aceeași valoare, aduce lizibilitate codului iar, în cazul în care valoarea ei trebuie schimbată, acest lucru se va produce într-un singur loc în program.

3.5 Tipurile de date numerice

Java are 8 tipuri de date primitive pentru valorile numerice, caracter și boolean (a se vedea Tabelul 1).

Tabelul 1. Tipuri de date primitive.

Tip	Interval de valori	Dimensiune
`byte`	-27 (-128) până la 27 -1 (127)	8 biți
`short`	-215 (-32768) până la 215 - 1 (32767)	16 biți
`int`	-231 până la 231 - 1	32 biți
`long`	-263 până la 263 - 1	64 biți
`float`	Negative: -3.4E+38 pana la -1.4E-45 Pozitive: 1.4E-45 pana la 3.4 E+38	32 biți IEEE 754
`double`	Negative: -1.79E+308 pana la -4.9E-324 Pozitive: 4.9E-324 pana la 1.79E+308	64 biți IEEE 754

Când se încearcă să se stocheze o valoare care se apropie de limita maximă a intervalului tipului de date, se produce un *overflow*. Trebuie avut grijă, întrucât Java nu generează avertizări sau erori referitoare la acest lucru. Pe de altă parte, atunci când se stochează o valoare în virgulă mobilă foarte mică (foarte aproape de zero), se produce *underflow*. Aici nu se solicită atenția programatorului deoarece Java aproximează aceste valori foarte mici la `0`.

3.6 Operatorii numerici

Operatorii numerici sunt `+`, `-`, `/`, `*`, și `%`. Când ambii operanzi sunt numere întregi, rezultatul operației de împărțire este un întreg. De exemplu `5 / 2 = 2`. Dacă unul dintre operanzi este număr real, atunci rezultatul este și el un număr real. De exemplu `5.0 / 2 = 2.5`. Operatorul `%` poate fi folosit și pentru numere negative: `-5 % 2 = -1`.

Acești operatori pot fi combinați cu operatorul de atribuire =, rezultând: `+=, -=, /=, *=` și `%=`. Asemănător limbajului C, avem și operatorii de incrementare prefixați și postfixați `++` și `--`.

3.7 Conversii de tip

Când se incearcă executarea unei operații în care avem un număr întreg și un număr real, Java convertește în mod automat numărul întreg la tipul real. Spre exemplu, `2 * 5.0` va fi echivalent cu `2.0 * 5.0`. Se poate atribui o valoare unei variabile care acoperă o plajă mai largă de valori. În sens invers nu se poate, atâta timp cât nu se folosește operația de conversie de tip (eng., *„type casting"*). Există două tipuri de casting: implicit la un tip mai general (eng., *„widening"* – se realizează automat, fără a se preciza acest lucru explicit la trecerea de la un tip de date mai specific la unul mai gneral) și explicit la un tip mai specific (eng., *„narrowing"* – trebuie obligatoriu precizat).

Exemplu widening:

```
int numarIntreg = 9;
double numarReal = numarIntreg;        // conversie automată
System.out.println(numarIntreg);       // Se va afisa 9
System.out.println(numarReal);         // Se va afisa 9.0
```

Exemplu narrowing:

```
int x = (int) 1.5;
```

În exemplul de mai sus, valoarea `double 1.5` este convertită la tipul `int`, prin urmare se va trunchia la partea întreagă, deci va fi egală cu `1` și se va atribui variabilei `x`. Acesta este un caz de *narrowing casting* ce trebuie explicitat în mod obligatoriu prin plasarea tipului între paranteze.

3.8 Tipul caracter

Tipul `char` denotă un singur caracter, reprezentat pe `16` biți, ce trebuie încadrat între apostroafe. Modalitatea de codificare a caracterelor în limbajul Java poartă denumirea de Unicode, fiind o schemă de codificare dezvoltată de Unicode Consortium pentru procesarea textelor în diverse limbaje de programare. S-a observat că cele `65 536` caractere posibile în reprezentarea pe `16` biți nu sunt suficiente pentru a reprezenta toate caracterele posibile. Standardul Unicode a fost extins până la `1112064` caractere. Caracterele Unicode sunt exprimate în hexazecimal de la `'\u0000'` la `'\uFFFF'`, și includ caracterele din standardul ASCII. Câteva exemple de conversii la tipul `int` sunt exemplificate în continuare.

```
char ch = (char) 66.9;
System.out.println(ch);
```

În exemplul de mai sus se va afișa valoarea `'B'`.

```
int i = '0' + '1';
System.out.println(i);
```

Va afișa valoarea `95` deoarece lui '`0`' îi corespunde codul `47`, iar lui '`1`' îi corespunde codul `48`.

```
int i = (int) 'a';
System.out.println(i);
```

Va afișa valoarea `97`.

3.9 Tipul String

Pentru a reprezenta mai multe caractere se folosește tipul `String`. Atunci când se atribuie o valoare de tipul `String` unei variabile, ea trebuie încadrată între ghilimele. `String` este de fapt o clasă predefinită în limbajul Java, alături de `System`, `Scanner` etc. `String` nu este un tip de bază sau primitiv, ci este cunoscut ca un *tip de referință*. Cel mai des întâlnit operator cu care lucrează stringurile este operatorul de concatenare `+` (cu variația `+=`). Asfel, pot fi concatenate atât două string-uri, cât și un string cu o valoare non-string. Cea din urmă este automat convertită în acest caz la tipul `String`.

3.10 Instrucțiuni

În limbajul Java, instrucțiunea de decizie `if` are aceeași formă ca în C sau C++, cu diferența că evaluarea unei expresii conduce la valorile de adevăr `true` sau `false`, care sunt de tipul `Boolean`. Secvențele de mai jos sunt echivalente:

```
if(x%2 == 0)
      par = true;
else
      par = false;
```

Echivalentă cu:

```
boolean par = x % 2 == 0
```

Operatorii logici sunt `!` (NOT), `&&` (ȘI), `||` (SAU) și `^` (SAU EXCLUSIV).

Similar se întâmplă cu instrucțiunea `switch` și cu cea **condițională**, care utilizează operatorul ternar `?:`

```
Expresie_booleană ? expresie1 : expresie2;
max = (a > b) ? a : b;
System.out.println((x%2 == 0) ? "par" : "impar");
```

Instrucțiunile repetitive din limbajul Java sunt `while`, `do while` și `for`, cu formă similară celei din limbajele C sau C++. Pentru a încheia iterația curentă și a trece automat la următoarea se folosește intrucțiunea `continue`, iar pentru a încheia ciclul repetitiv se utilizează `break`.

3.11 Afișarea datelor cu format

Pentru afișarea datelor cu format se utilizează metoda `System.out.printf`, care are forma:

```
System.out.printf(format, item_1, item_2, ... item_n);
```

Cei mai des întâlniți specificatori de format sunt `%b` – `Boolean`, `%c` – caracter, `%s` – `String`, `%d` – întreg, `%f` – număr real, `%e` – real în notație științifică.

Exemple de specificatori de format cu precizie:

- `%4c` – afișează caracterul și lasă `3` spații înaintea lui.
- `%5d` – afișează numărul întreg de dimensiune cel puțin `5`. Dacă numărul de cifre este mai mic decât `5`, se adaugă spații înainte. Dacă numărul de cifre este mai mare decât `5`, dimensiunea lui se mărește automat.
- `%10.2f` – afișează numărul în virgulă mobilă, de dimensiune cel puțin `10`, incluzând virgula și două cifre după virgulă. Deci, sunt `7` cifre alocate înainte de virgulă. Dacă numărul de cifre de dinainte de virgulă este mai mic de `7`, se adaugă spații înainte de număr. Dacă numărul de cifre este mai mare decât `7`, dimensiunea este automat incrementată.
- `%12s` – dacă dimensiunea șirului de caractere este mai mică de `12`, se adaugă spații înaintea lui, altfel se suplimentează dimensiunea.

Tipul datelor afișate trebuie să fie compatibil cu specificatorii. Dacă vrem să afișăm valoarea reală `40.0`, trebuie să folosim `%f` sau `%e`. Dacă încercăm să afișăm valoarea `40`, nu putem folosi `%f` sau `%e`.

Semnul `%` denotă un specificator. Dacă doriți să afișați literalul `%`, folosiți `%%`.

3.12 Precedența și asociativitatea operatorilor

Precedența și asociativitatea operatorilor determină ordinea în care aceștia sunt evaluați. Când doi operatori cu aceeași precedență sunt alăturați, asociativitatea determină ordinea evaluării. Toți operatorii binari sunt asociativi stânga, cu excepția operatorului de atribuire care are asociativitate dreapta.

`a = b += c = 5` este echivalentă cu `a = (b += ( c = 5) )`

Dacă inițial `a = 1`, `b = 1` și `c = 1`, după evaluarea expresiei, `a = 6`, `b = 6` și `c = 5`.

4 Aplicații

1. Compile and run the *Hello, World!* Application from the terminal/command line. You need a compiler (you may use javac) and Java available via terminal to run the program.

Expected commands:

```
javac MyHello.java
```

```
java MyHello
```

Expected output:

```
Hello, World!
```

2. Install different versions of JDK on your machine. Link them into your IDE (be it IntelliJ IDEA, Netbeans, or another one) and try to compile and run the *Hello, World!* application using different JDK versions.

3. Consider the following code:

```
boolean yes = true;
boolean no = false;
int loVal = -999;
int hiVal = 999;
double grade = 87.5;
double amount = 50.0;
String hello = "world";
```

What is the result of the expressions listed in the table?

Expression	**Result**
yes == no || grade > amount	
amount == 40.0 || 50.0	
hiVal != loVal || loVal < 0	
true || hello.length() > 0	
hello.isEmpty() && yes	
grade <= 100 && !false	
!yes || no	
grade > 75 > amount	
amount <= hiVal && amount >= loVal	
no && !no || yes && !yes	

4. What do the following lines print?

public class Demo {

```
    public static void main(String[] args) {
        float a = 100.25f;
        long b = (long)a;
        System.out.println("value of a: "+a);
        System.out.println("value of b:"+b);
        int c = (int)b;
        System.out.println("value of c:"+c);
        byte d = (byte)c;
        System.out.println("value of d:"+d);
    }
}
```

5. What do the following lines print?

```
int a = 5;
System.out.println(a + -a - a++ % 10);
System.out.println(a - a + --a / 10);
for (int i = 2; i < 5;) {
        i++;
        a += a;
}
System.out.println("a = " + a);
```